

Emporix Punchout Plugin — Design Spec

Emporix Punchout Plugin — Design Spec

Date: 2026-05-11 **Status:** Approved

Overview

A Node.js/TypeScript Emporix Marketplace plugin that transforms an Emporix-powered supplier store into a punchout-enabled catalog. Enterprise buyers authenticate from their procurement systems (Ariba, Coupa, Jaggaer, Workday, Oracle, SAP SRM/S4HANA), shop the Emporix storefront, and return a filled cart to their procurement system — without creating an Emporix order.

Protocols supported at launch: cXML 1.2 and OCI 4.0.

Architecture

Single Express/TypeScript service deployed as a Docker container. Stateless beyond a Redis session store.

Internal modules

Module	Responsibility
protocols/cxml	cXML parsing, validation, response generation
protocols/oci	OCI form parsing and response generation
emporix/client	Emporix REST API wrapper (catalog, cart, price lists, customer groups)
session/store	Redis-backed punchout session state
admin/router	

Module	Responsibility
	Admin API routes + Emporix JWT verification middleware

Endpoint groups

Public punchout endpoints (called by procurement systems, no Emporix auth required):

- POST /punchout/cxml/setup — receives PunchOutSetupRequest, returns PunchOutSetupResponse with one-time session URL
- POST /punchout/oci/setup — receives OCI form POST, returns HTML redirect to session URL
- GET /session/:token — validates token, creates Emporix guest cart, redirects to storefront
- POST /punchout/return — protocol-aware return endpoint; reads session protocol field and routes to cXML (PunchOutOrderMessage → BrowserFormPost URL) or OCI (field POST → HOOK_URL) handler

Admin extension endpoints (called by Emporix dashboard iframe, protected by Emporix JWT):

- GET/POST /admin/config — merchant reads/writes plugin configuration
- GET/POST /admin/buyers — merchant manages buyer org → customer group mappings

Protocol Flows

cXML Punchout

1. Procurement system sends PunchOutSetupRequest to POST /punchout/cxml/setup with BuyerCookie, credentials, and BrowserFormPost return URL
2. Plugin validates shared secret (HMAC), extracts buyer org identity, resolves to Emporix customer group via buyer mapping table
3. Plugin creates punchout session in Redis with buyer org, customer group ID, BrowserFormPost URL, and a one-time token (15-minute TTL)
4. Plugin returns PunchOutSetupResponse with StartPage URL: https://<plugin-host>/session/<token>
5. Browser hits /session/<token>: token validated and deleted (single-use), Emporix guest cart created scoped to customer group, session cookie set, redirect to storefront
6. Buyer shops in Emporix storefront
7. Buyer clicks “Return Cart” — browser POSTs to POST /punchout/return with cart ID (routes internally to cXML or OCI handler based on session protocol field)
8. Plugin fetches live cart from Emporix, translates each line item to a cXML ItemIn, wraps in PunchOutOrderMessage, HTTP-POSTs to BrowserFormPost URL

9. PunchOutOrderMessage operationAllowed set to edit (buyer may modify requisition after return)

OCI Punchout

Same flow with protocol-specific differences: - Setup arrives as form POST with fields HOOK_URL, USERNAME, PASSWORD, ~OkCode, ~Caller - Plugin returns HTML redirect (not XML) to session start URL - Return posts OCI-formatted fields (NEW_ITEM-DESCRIPTION[n], NEW_ITEM-MATNR[n], etc.) to saved HOOK_URL - ~OkCode return field defaults to ADDFROMCATALOG

Session Expiry Handling

Two-tier TTL strategy: - One-time token: 15-minute TTL, deleted on first use (prevents replay attacks) - Full session record: 2-hour TTL (covers expected shopping window)

Token expired before storefront load: /session/<token> returns a user-facing HTML error page: “Your punchout session has expired. Please return to your procurement system and try again.”

Session lost during shopping: On return request, if session lookup fails, plugin returns a protocol-appropriate error page. No cart recovery attempted — buyer must restart punchout from their procurement system.

Multi-Item Quantity Changes

- Plugin always fetches **live cart state** from Emporix at return time — quantity changes and removals are automatically reflected
 - Removed items simply do not appear in the returned payload
 - operationAllowed=edit signals to procurement systems that the buyer may further edit the requisition post-return
 - OCI uses ADDFROMCATALOG (safe default for multi-item edits) rather than BACKGROUND_DELETE
-

Emporix Integration

APIs used

API	Usage
Cart API	Create guest cart scoped to customer group at session start; read cart on return
Customer Group API	

API	Usage
	Populate buyer mapping dropdown in admin UI
Price List API	Not called directly — Emporix applies price list rules automatically when cart is created under a customer group
Catalog API	Read product data (SKU, name, description, UOM) for line item translation
OAuth2 / Service Account	Plugin authenticates via service account; buyer-facing requests use guest cart token
Configuration Service	Store plugin config per tenant

Buyer org → customer group mapping

Stored in plugin config as an array of { buyerOrgId, customerGroupId } pairs. Resolved at setup time using the buyer org identity from the cXML BuyerCookie or OCI USERNAME. If no mapping exists, falls back to Emporix default customer group (standard catalog prices).

Storefront integration

The plugin does not replace or fork the Emporix storefront. It: 1. Creates the guest cart and sets a session cookie before redirecting to the storefront 2. Serves a small JS snippet (/punchout-widget.js) that merchants add to their storefront <head>. The snippet detects the punchout session cookie and renders a “Return Cart” button only during active punchout sessions.

Admin UI Extension

Registered via plugin.json manifest. Rendered as an iframe in the Emporix merchant dashboard. Emporix passes a signed JWT in the URL hash; the admin frontend uses it to authenticate all calls to /admin/* endpoints.

Screens:

1. **Credentials** — shared secret, Emporix service account (client ID + secret), storefront base URL
2. **Buyer Mappings** — table of buyer org ID → customer group (add/edit/delete); customer group dropdown populated live from Emporix API
3. **Protocol Settings** — enable/disable cXML and OCI; operationAllowed mode; OCI ~0kCode behaviour
4. **Connection Test** — send a simulated punchout setup request and inspect the parsed response to verify config before going live

Static bundle served by the plugin service itself. All /admin/* routes verify Emporix JWT signature via Emporix’s public JWKS endpoint.

Data Model

Redis key schema

punchout:session:<token> # one-time token → session ID (15min TTL, deleted on use)
punchout:active:<sessionId> # full session record (2hr TTL)

Session record

```
{  
  sessionId: string  
  protocol: 'cxml' | 'oci'  
  buyerOrgId: string  
  customerGroupId: string  
  browserFormPostUrl: string  
  buyerCookie: string  
  emporixCartId: string | null  
  createdAt: string // ISO8601  
}
```

Plugin configuration (per Emporix tenant)

```
{  
  sharedSecret: string // bcrypt-hashed  
  serviceAccount: {  
    clientId: string  
    clientSecret: string // AES-256 encrypted at rest  
  }  
  storefrontBaseUrl: string  
  cxmlEnabled: boolean  
  ociEnabled: boolean  
  operationAllowed: 'create' | 'edit' | 'inspect'  
  ociOkCode: 'ADDFROMCATALOG' | 'SOURCING'  
  buyerMappings: Array<{  
    buyerOrgId: string  
    customerGroupId: string  
  }>  
}
```

Error Handling & Security

Incoming validation: - cXML shared secret validated via HMAC on every setup request — reject with cXML 400 status if invalid - OCI credentials validated against stored shared secret — return HTTP 401 if mismatch - All payloads parsed with strict XML/form schemas; malformed input never surfaces raw Express errors

Outbound error handling: - Emporix API failures during session start return a user-facing HTML error page to the procurement iframe - Emporix API failures during return send a protocol-appropriate error to the procurement system

Security hardening: - Shared secrets stored bcrypt-hashed, never logged or returned via admin API - Service account credentials AES-256 encrypted at rest - Punchout endpoints rate-limited per source IP (express-rate-limit) - Session tokens are single-use UUIDs - Admin routes enforce Content-Security-Policy headers - All outbound HTTP calls enforce a 5-second timeout (configurable)

Testing Strategy

Layer	Scope
Unit	cXML and OCI protocol parsers/generators; edge cases (empty carts, special characters, malformed XML)
Integration	Full setup → session → return flow with mocked Emporix API (MSW/nock) and real Redis test instance
Contract	cXML responses validated against cXML 1.2 DTD; OCI fields validated against OCI 4.0 spec
Admin API	JWT middleware, config CRUD, buyer mapping CRUD
E2E smoke	Scripted cXML setup request against running plugin instance, simulating Ariba-style procurement system

Out of Scope (v1)

- Order creation in Emporix at punchout return time
- xCBL protocol support
- Third-party eProcurement hub integration
- Custom pricing logic outside Emporix native price lists
- Inventory reservation during punchout session